



State Machine Diagrams

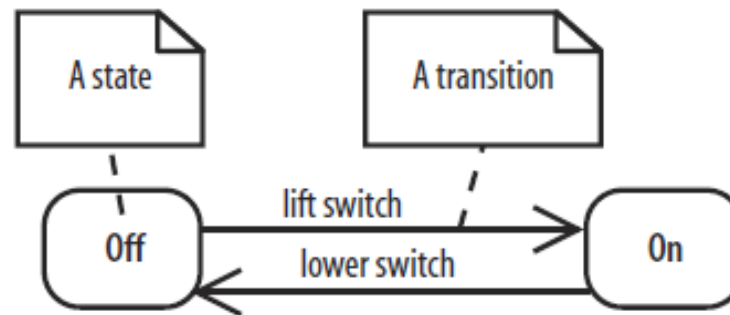


Francesco Guerra
francesco.guerra@unimore.it

State Machine Diagrams

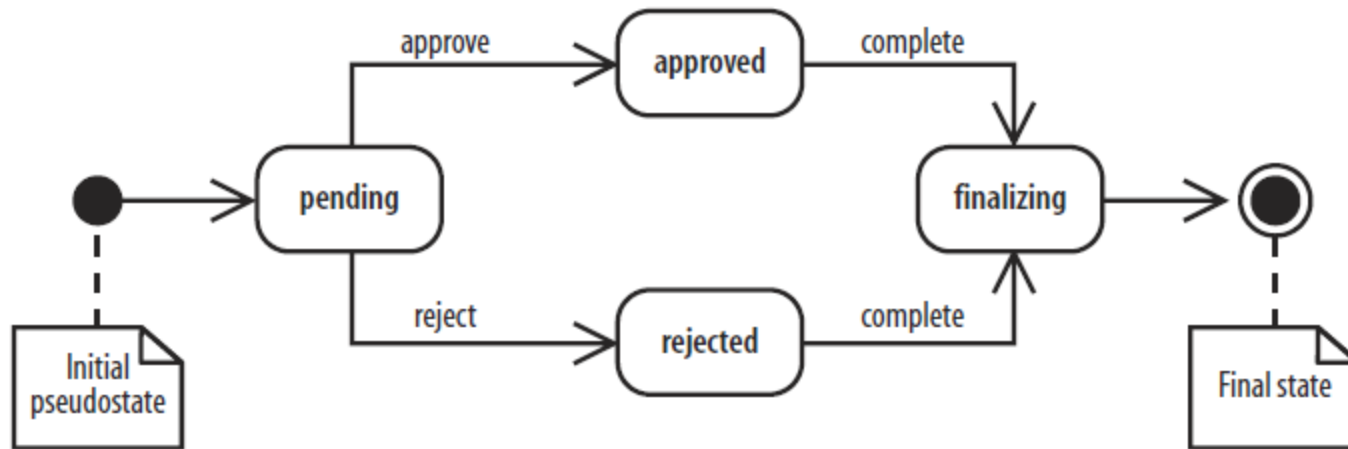
- ▶ Activity diagrams are useful for describing behavior, but there's a missing piece.
- ▶ Sometimes the state of an object or system is an important factor in its behavior.
 - ▶ For example, if the CMS required potential users to submit an application for an account, which could be approved or rejected, then the AccountApplication object may act differently depending on whether it is pending, accepted, or rejected.
- ▶ In such situations, it's helpful to model states of an object and the events causing state changes—this is what state machine diagrams do best.
 - ▶ Continuing the above example, the AccountApplication object could have the states pending, accepted, and rejected as possible values of an attribute, and change states upon events such as approve or reject. A state machine diagram allows you to model this behavior.
- ▶ State machine diagrams are heavily used in special niches of software and hardware systems, including the following:
 - ▶ Real-time/mission-critical systems, such as heart monitoring software
 - ▶ Dedicated devices whose behavior is defined in terms of state, such as ATMs
 - ▶ First-person shooter games,...

Essentials



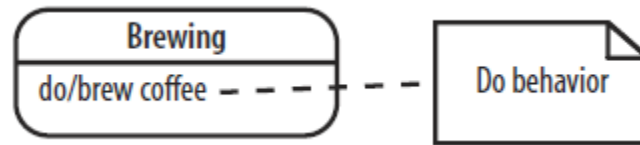
- ▶ The fundamental elements of a state diagram: states and transitions between states
- ▶ A state is *active* when entered through a transition, and it becomes *inactive* when exited through a transition
- ▶ The event causing the state change, or *trigger*, is written along the transition arrow

Essentials (2)



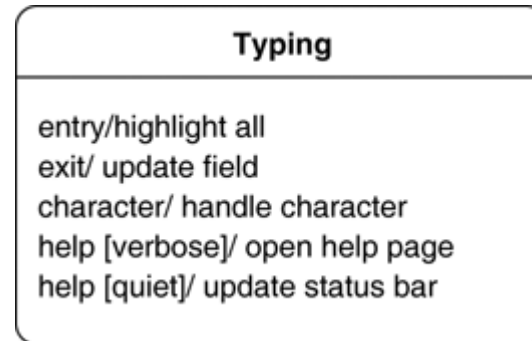
- ▶ State diagrams usually have an *initial pseudostate* and a *final state*, marking the start and end points of the state machine

States



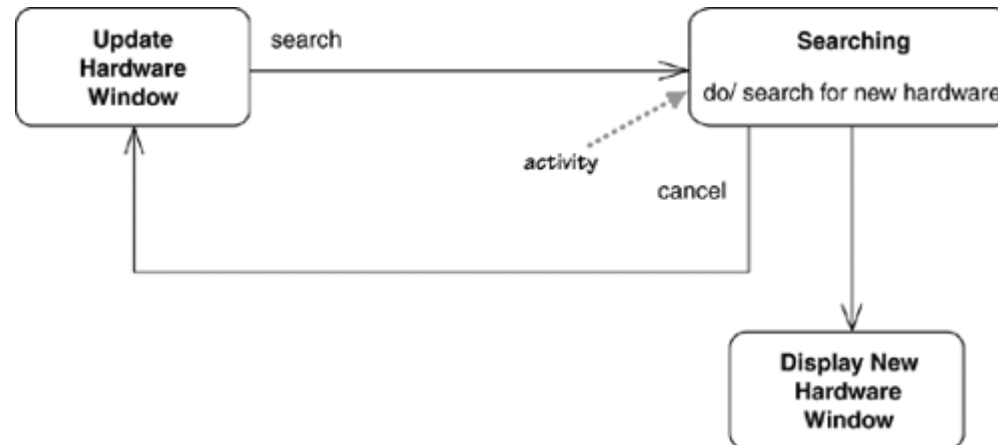
- ▶ A *state* is a condition of being at a certain time.
 - ▶ A state can be a passive quality, such as On and Off for the light object.
 - ▶ A state can be an active quality, or something that an object is doing.
 - ▶ For example, a coffeemaker has the state Brewing during which it is brewing coffee.
- ▶ A state is drawn as a rounded rectangle with the name of the state in the center
- ▶ In a “doing” state, you can write the behavior inside the state
 - ▶ Do behavior, written as *do/behavior*, is behavior that happens as long as the state is active.
 - ▶ Do behavior either completes on its own or is forced to complete when a trigger causes the state to exit

Internal Activities



- ▶ States can react to events without transition, using internal activities:
 - ▶ putting the event, guard, and activity inside the state box itself.
- ▶ two special activities:
 - ▶ The entry activity is executed whenever you enter a state;
 - ▶ the exit activity, whenever you leave.

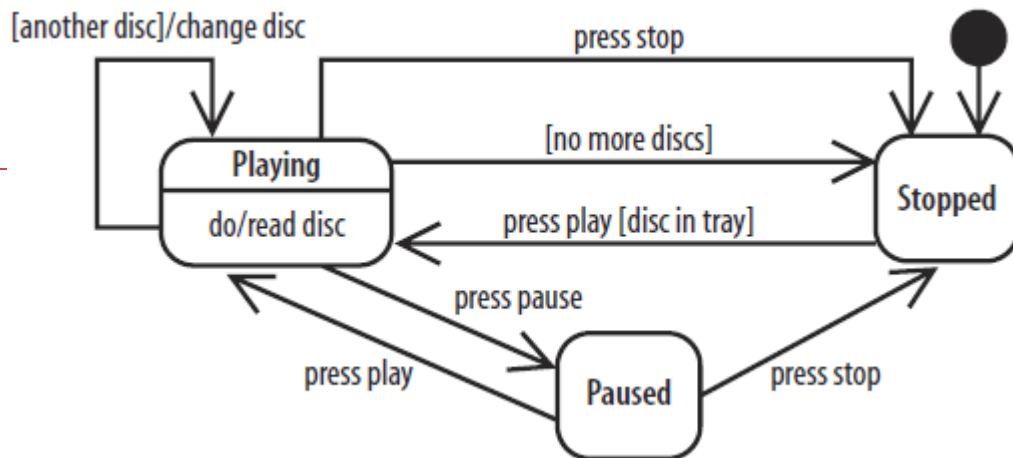
Activity States



- ▶ You can have states in which the object is doing some ongoing work.
 - ▶ The ongoing activity is marked with the **do/**; hence the term **do-activity**.
 - ▶ Once the search is completed, any transitions without an activity are taken.
 - ▶ If the **cancel** event occurs the **do-activity** is halted.
- ▶ Both **do-activities** and regular activities represent carrying out some behavior.
- ▶ The difference between the two is that regular activities occur “instantaneously” and cannot be interrupted by regular events, while **do-activities** can take finite time and can be interrupted.
- ▶ UML 1 used the term **action** for regular activities and used **activity** only for **do-activities**.

Transitions

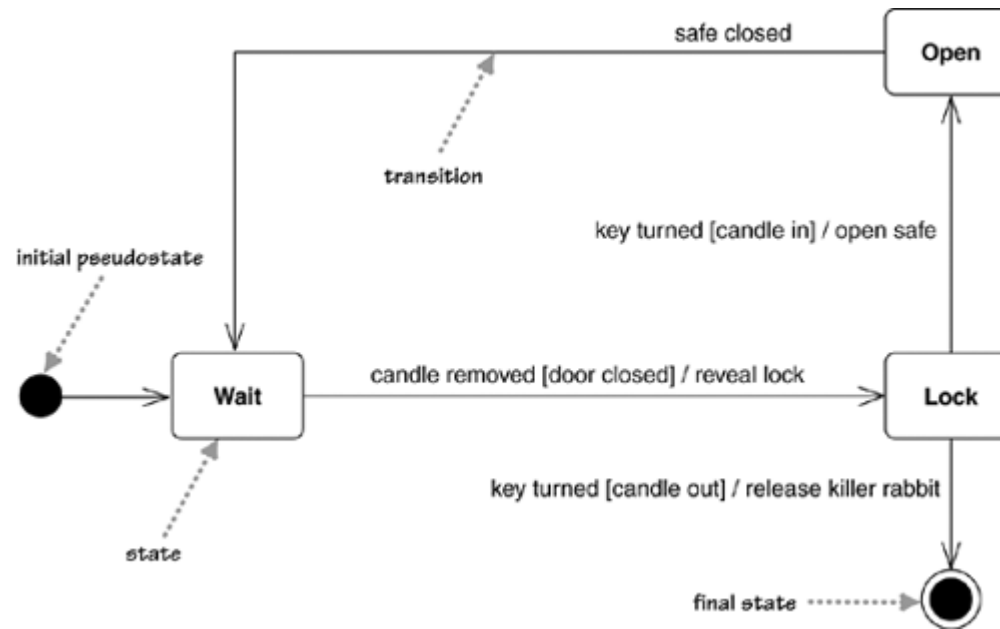
- ▶ The transition indicates a movement from one state to another.
 - ▶ Each transition has a label that comes in three parts: trigger-signature [guard]/activity.
 - ▶ The trigger-signature is usually a single event that triggers a potential change of state.
 - ▶ The guard, if present, is a Boolean condition that must be true for the transition to be taken.
 - ▶ The activity is an uninterruptible activity that executes while the transition takes place (if the transition is taken).
- ▶ All three parts to a transition are optional.
 - ▶ A missing activity indicates that you don't do anything during the transition.
 - ▶ A missing guard indicates that you take the transition if the event occurs.
 - ▶ A missing trigger-signature indicates that you take the transition immediately.



Transitions (2)

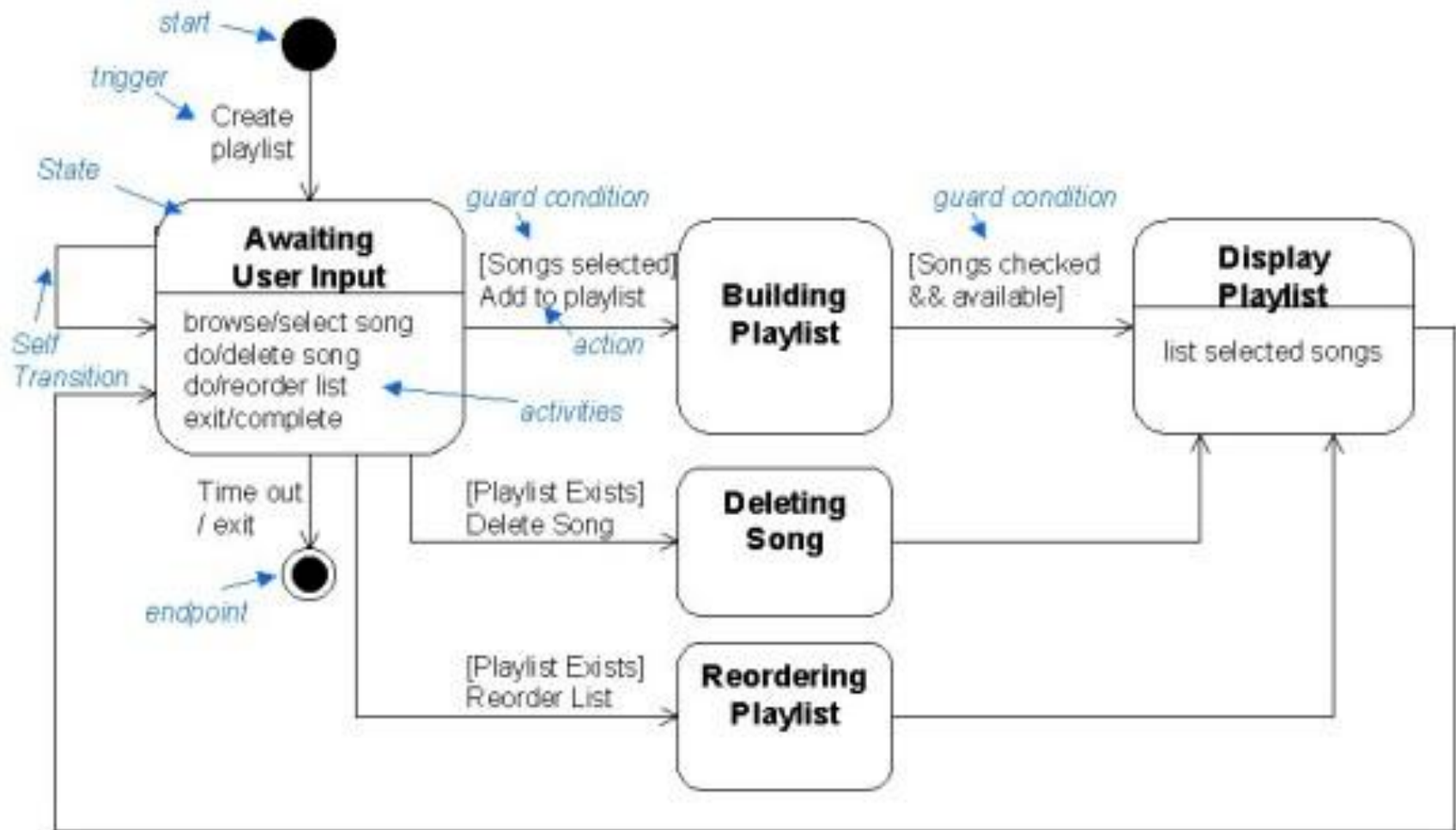
- ▶ When an event occurs in a state, you can take only one transition out of it.
 - ▶ If you use multiple transitions with the same event, the guards must be mutually exclusive.
 - ▶ If an event occurs and no transition is valid, the event is ignored.
- ▶ The final state indicates that the state machine is completed, implying the deletion of the controller object
- ▶ State machines can show only what the object directly observes or activates.
- ▶ When developers talk about objects, they often refer to the state of the objects to mean the combination of all the data in the fields of the objects.
 - ▶ However, the state in a state machine diagram is a more abstract notion of state;
 - ▶ Essentially, different states imply a different way of reacting to events.

Example



- ▶ State machine diagrams describe the behavior of a system.
 - ▶ The controller can be in three states: Wait, Lock, and Open
 - ▶ The diagram gives the rules by which the controller changes state.
 - ▶ These rules are in the form of transitions: lines that connect the states.

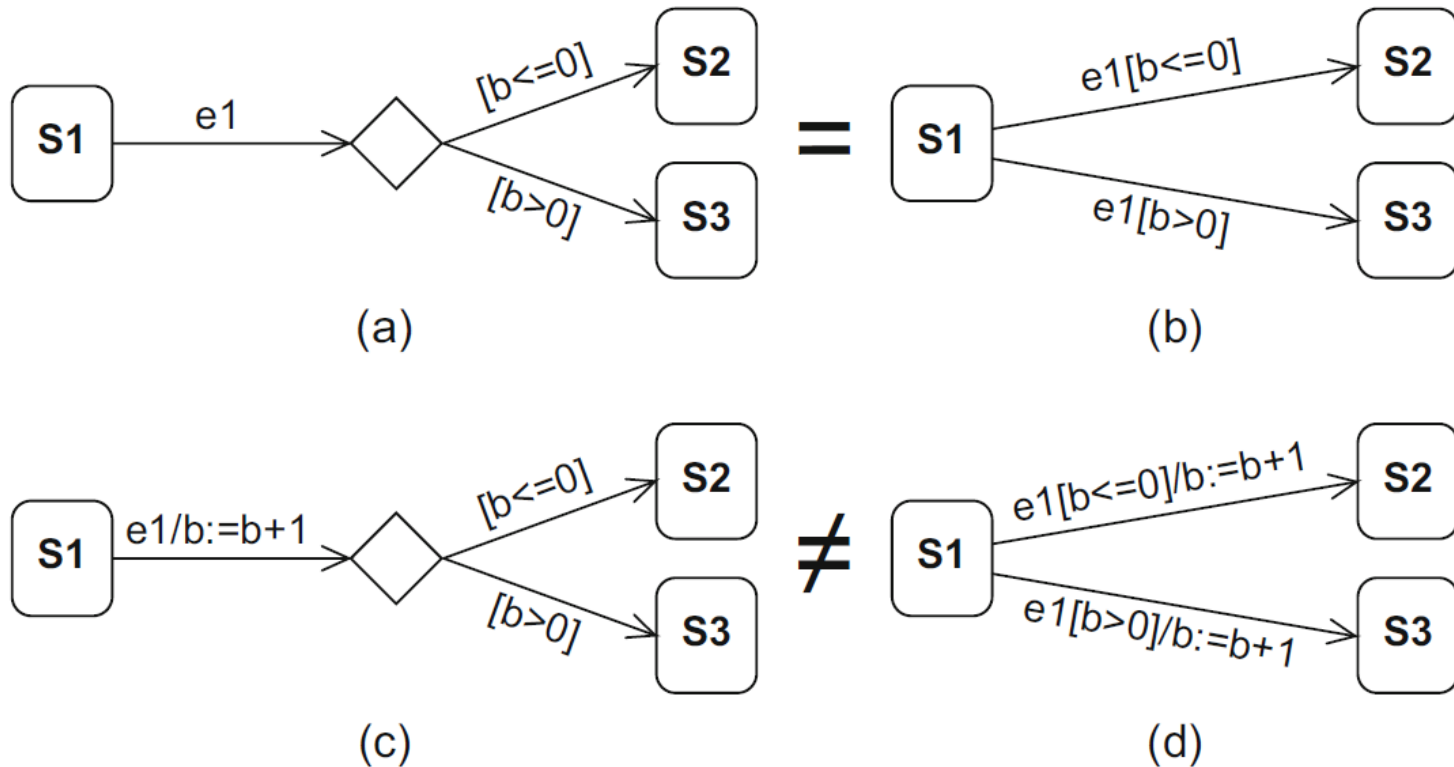
Example (2)



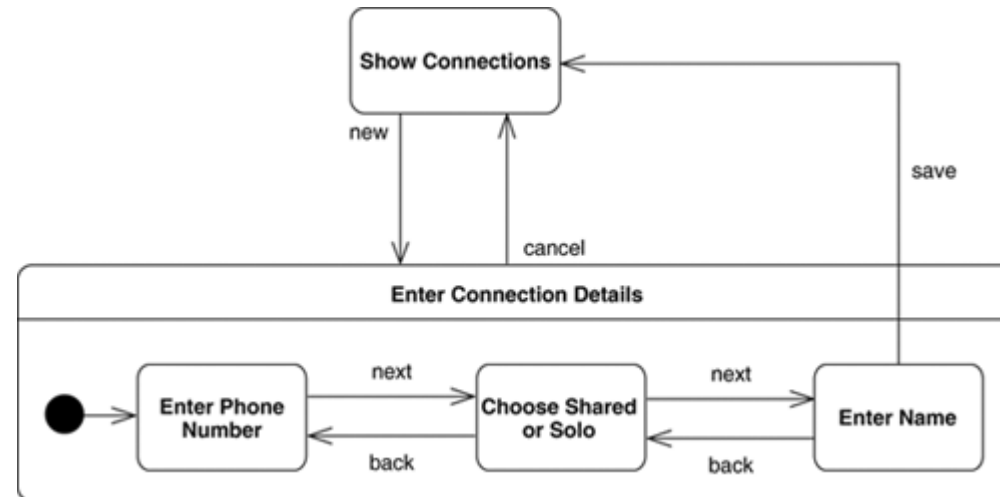
Decision nodes

- ▶ The *decision node* is represented in the diagram with a diamond.
 - ▶ You can use it to model alternative transitions.
 - ▶ At the incoming edge, you model the event that triggers the transition;
 - ▶ At the outgoing edges, you specify the guards for the alternative paths for the state transition.
 - ▶ You can also specify activities at the incoming edge and all outgoing edges.
 - ▶ If the event modeled at the incoming edge occurs, the system enters the transition.
 - ▶ It evaluates the guards and thus select the outgoing edge to be used.
 - ▶ To prevent the system getting “stuck” in the decision node, you must ensure that the guards cover all possible situations.
 - ▶ Using [else] at one of the edges will allow you to do this.
 - ▶ If the guards are not mutually exclusive, and if two or more edges are evaluated as true, one of these valid edges is selected nondeterministically.

Example



Superstates



- ▶ You'll find that several states share common transitions and internal activities.
- ▶ In these cases, you can make them substates and move the shared behavior into a superstate.
- ▶ Without the superstate, you would have to draw a cancel transition for all three states within the Enter Connection Details state.

Types of events

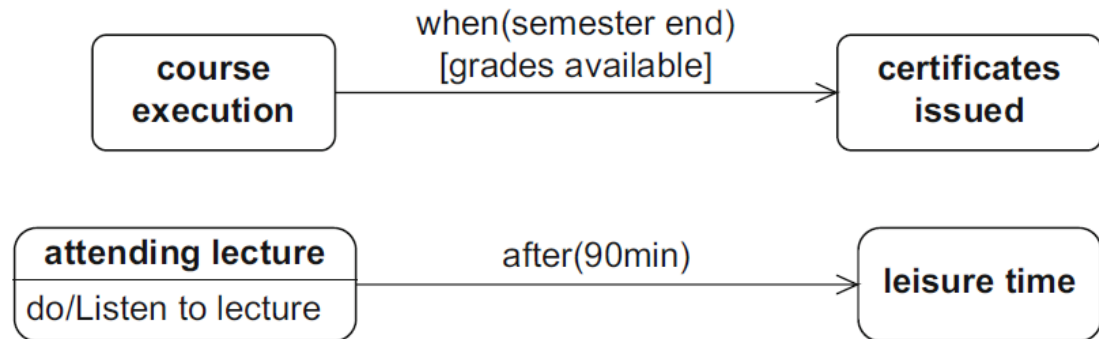
- ▶ The most important types of events defined in UML are the signal event, call event, time event, change event, any receive event, and completion event.
- ▶ The *signal event* is used for asynchronous communication.
 - ▶ A sender sends a signal to a receiver and does not wait for an answer.
 - ▶ The receiver is the modeled object and the sender can be another object or the same object as the receiver.
 - ▶ The receipt of the signal is processed as an event.
 - ▶ The name of the event corresponds to that of the signal and arguments can be specified.
 - ▶ For example, rightmouse- down or send sms(message)
- ▶ *Call event*: *Call events* are operation calls.
 - ▶ The name of the event corresponds to the name of an operation including parameters,
 - ▶ For example, occupy(user,lectureHall) or register(exam).

Types of events

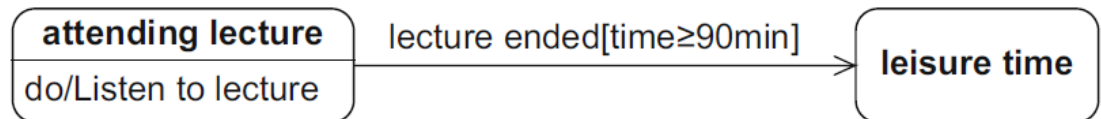
- ▶ *Time event*: *Time events* enable time-based state transitions.
 - ▶ The specified time can be relative or absolute.
 - ▶ for example, after(5 seconds), when(time==16:00) or when(date==20150101)
- ▶ *Change event*: You can use a *change event* to permanently monitor whether a condition becomes true.
 - ▶ A change event consists of a boolean expression and the preceding keyword when.
 - ▶ Examples of change events are when(registrations==number of seats) or when($x > y$).
- ▶ You can use an *any receive event* to specify a type of “else” transition.
 - ▶ the keyword all is attached to a transition as an event which occurs when any event occurs that does not trigger another transition for the active state.
- ▶ A *completion event* takes place when everything to be done in the current state is completed.
 - ▶ This includes entry and do activities as well as the completion of nested states, if there are any.
 - ▶ If a state has an outgoing transition without any event specified, the completion event triggers this transition.

Examples

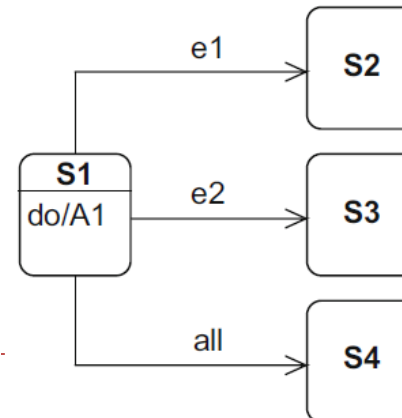
► Change events



► Change event modeled with guard



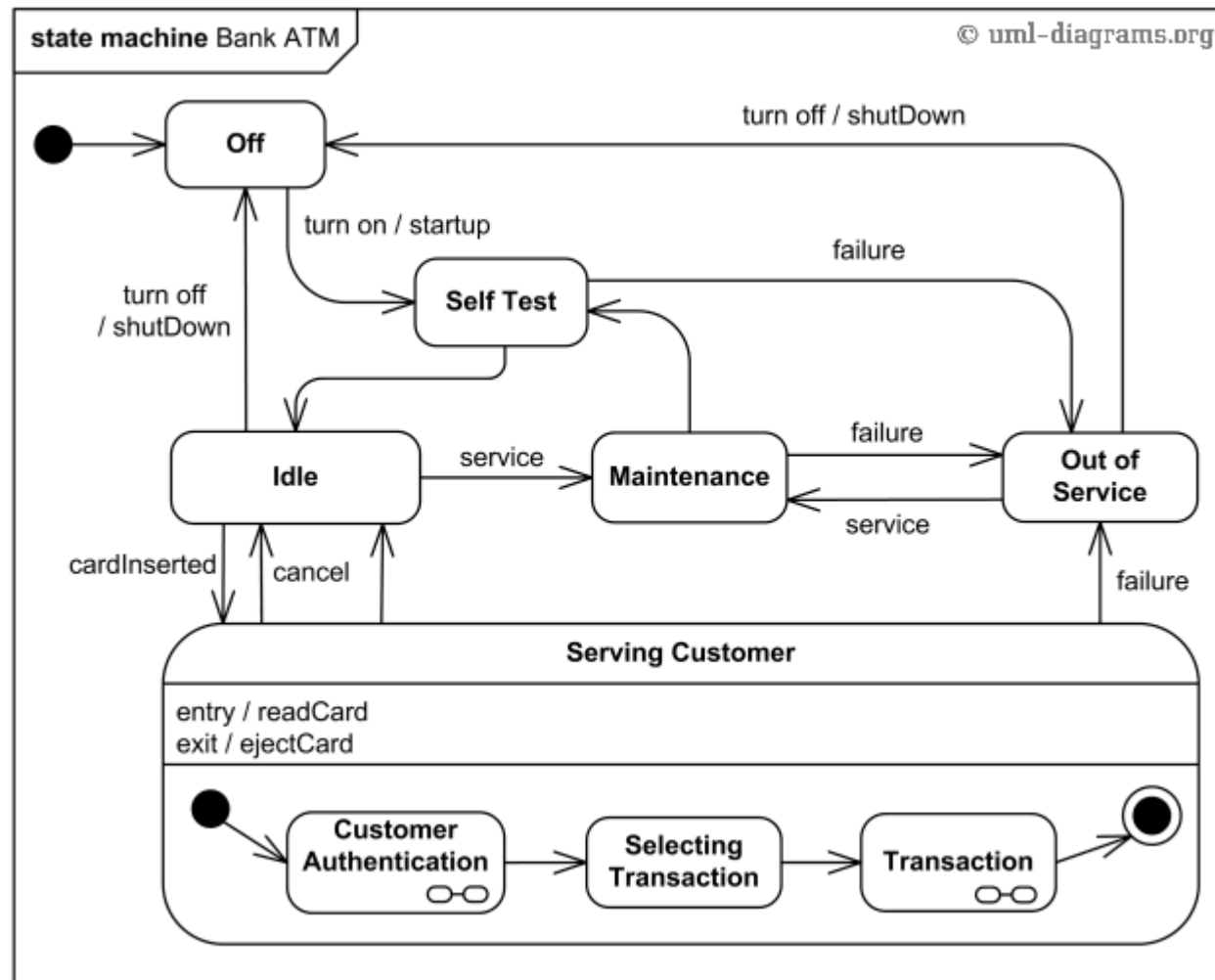
► Any receive event



When to Use State Diagrams

- ▶ State diagrams are good at describing the behavior of an object across several use cases.
- ▶ State diagrams are not very good at describing behavior that involves a number of objects collaborating.
 - ▶ Not everyone finds state diagrams natural.
 - ▶ Keep an eye on how people are working with them.
 - ▶ It may be that your team does not find state diagrams useful to its way of working.
- ▶ If you do use state diagrams, don't try to draw them for every class in the system.
 - ▶ Although this approach is often used, it is almost always a waste of effort.
 - ▶ Use state diagrams only for those classes that exhibit interesting behavior, where building the state diagram helps you understand what is going on.
 - ▶ Many people find that UI and control objects have the kind of behavior that is useful to depict with a state diagram.

Example

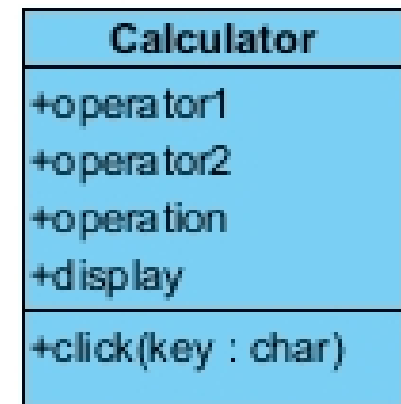


Esercizio

- ▶ Si disegni una macchina a stati per rappresentare tutti i possibili stati e relative transizioni in cui si può trovare un SMS da quando qualcuno inizia a scriverlo fino a quando deciderà di spedirlo. Si noti che una volta inviato il messaggio potrebbe non venire spedito immediatamente, ma in alcuni casi sono necessari alcuni tentativi prima che la spedizione vada a buon fine.

Esercizio

- ▶ Si disegni il diagramma relativo al funzionamento di una semplice calcolatrice
- ▶ L'interfaccia della calcolatrice si compone di 10 cifre e di 4 possibili operatori (+, -, *, /).
- ▶ Si ha inoltre il bottone "C" per cancellare il display e il bottone "=" per eseguire l'operazione



Liberamente tratto da:

References

- ▶ Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- ▶ Russ Miles, Kim Hamilton: Learning UML 2.0 A Pragmatic Introduction to UML, O'Reilly Media, 2006
- ▶ Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.